A Handwritten Music Recognition System Based on Visual Object Detection and semantic Assembly

Yuguang Yao Hok Seng

December 14, 2025

Abstract

Optical Music Recognition is a field of research that investigates how to computationally read music notation in documents [2]. This project proposes an end-to-end recognition pipeline, inspired by a precedent work [4]. We first reproduce the original system. The system leverages an advanced deep learning object detection model (YOLOv12) to extract basic musical symbols from score images, and combines a MLP module to identify the relationships between symbols with a constructed benchmark to evaluate the performance of the system. To realize a full OMR functionality, we expand this system with a rule-based assembly module to a ready-to-use musicXML file. Specifically, this assembly module resolves complex musical structure reconstruction issues, including multi-staff grouping, polyphonic alignment, and tuplet detection.

Keywords— Optical Music Recognition, YOLOv12, MLP, Scoretree, MusicXML

1 Introduction

Optical Music Recognition (OMR) is defined as a research field that investigates how to computationally read music notation in documents, aiming to invert the music encoding process to recover both musical semantics and structured encoding [2]. Unlike Optical Character Recognition (OCR), which typically deals with linear sequences of text, OMR must handle a complex visual language where the goal is often to enable replayability or detailed score editing. This requires the system not only to identify symbols but also to interpret their specific musical meaning, such as pitch and duration, which distinguishes OMR as a unique challenge separate from standard graphics recognition.

The primary difficulties in OMR stem from the two-dimensional and contextual nature of music notation, where the meaning of a symbol depends heavily on its spatial relationship with other elements, such as clefs and staff lines, rather than just its shape. Visual processing is further complicated by the extreme variability in symbol sizes, ranging from tiny dots to large braces, and the frequent superimposition or complex alignment of objects like beams and slurs. Recent work has shown that while Convolutional Recurrent Neural Networks (CRNN) can be applied to OMR in an end-to-end fashion, their application is not straightforward due to the presence of different elements sharing the same horizontal position, particularly in homophonic scores where multiple voices occur simultaneously [1]. Additionally, the field faces significant hurdles due to the lack of standardized evaluation metrics and output formats, making it difficult to objectively assess the performance of OMR systems or compare their ability to fully reconstruct a score.

Recent advances in end-to-end OMR have focused on bridging the gap between recognition outputs and practical music notation formats. Mayer et al. [3] proposed a linearized MusicXML encoding that maintains compatibility with industry-standard formats while enabling end-to-end training for pianoform music. Building on this direction, Yang et al. [4] presented a more complete OMR solution that combines deep learning object detection with graph-based relationship modeling, demonstrating the effectiveness of separating visual recognition from semantic assembly. Their work serves as a key inspiration for our approach.

Our project aims to solve this challenge by combining deep learning visual recognition with symbolic logical reasoning. We propose a multi-stage processing framework: visual perception, structure construction, semantic reconstruction, and digital generation. The core contributions of this system are:

1. Proposed a high-precision handwritten music symbol detection scheme based on **YOLOv12**.
2. Designed a MLP-based score semantic assembly algorithm capable of handling complex symbol dependencies.
3. Implemented a complete conversion flow from image to MusicXML, supporting polyphony and complex rhythms.

2 System Pipeline

The core architecture of this project consists of a sequential processing pipeline: Dataset Preparation, Preprocessing, Visual Object Detection, Relation Prediction, Semantic Assembly, and MusicXML Generation. Each stage is

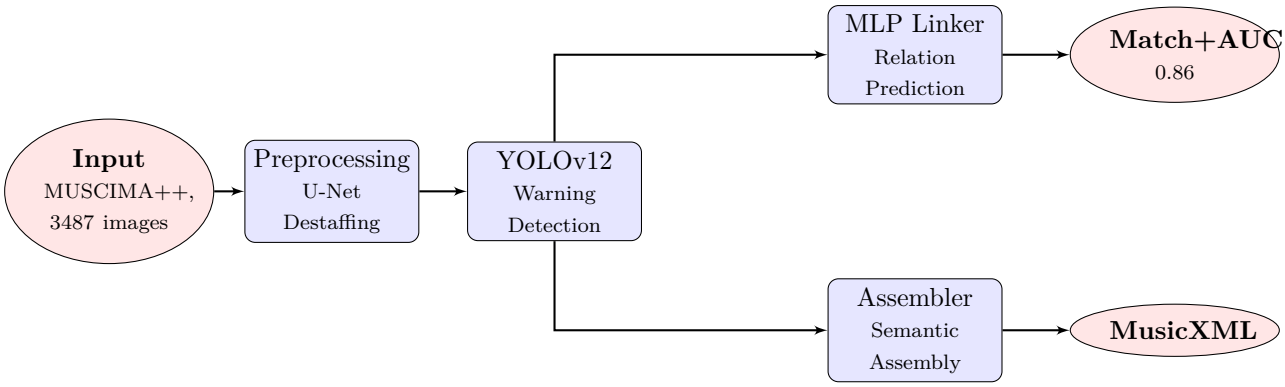

Figure 1: System Data Flow: The pipeline processes the input image through visual detection (YOLO), relation inference (MLP Linker), and rule-based assembly to generate the final MusicXML.

described below with implementation details and experimental results.

2.1 Dataset

We evaluate our system on the **MUSCIMA++** dataset, which contains 140 pages of complex handwritten music with 3487 cropped images. The dataset provides ground-truth annotations for both symbol bounding boxes and their relationships (Notation Graph). We follow the standard train/validation/test split logic to ensure rigorous evaluation. The dataset covers 117 symbol classes including noteheads, stems, beams, rests, accidentals, clefs, and time signatures.

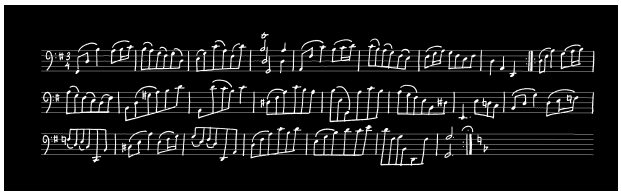

Figure 2: Example input image from MUSCIMA++ dataset showing handwritten musical notation.

2.2 Data Preprocessing

To optimize the input for object detection, we employ a **U-Net** architecture for image preprocessing. U-Net is a convolutional network with an encoder-decoder structure and skip connections, originally designed for biomedical image segmentation. In our pipeline, it is trained to perform pixel-wise classification, effectively separating musical symbols from staff lines and background noise.

We processed the raw data as follows:

1. **Staff Removal**: Leveraging the U-Net output, we remove the staff lines that often overlap with and obscure musical symbols. This "destaffing" significantly improves the signal-to-noise ratio for the subsequent YOLO detector.

2. **Image Cropping**: High-resolution full-page scores are sliced into local patches suitable for YOLO model input (typically 640×640). We employed an overlapping strategy during slicing to prevent key symbols from being cut off at the boundaries.

3. **Annotation Conversion**: MUSCIMA++ XML notation graph annotations were converted into YOLO format for training.

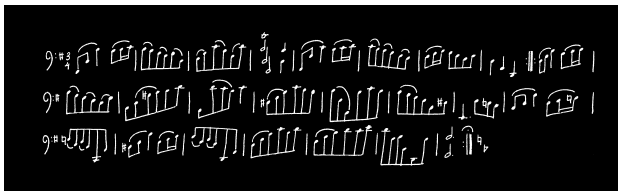

Figure 3: Result after U-Net preprocessing: staff lines removed, symbols preserved.

2.3 Visual Object Detection (YOLOv12)

We adopt the **YOLO (You Only Look Once)** series models as the visual perception engine. The project utilizes the latest **YOLOv12** architecture. Compared to YOLOv8, v12 significantly improves detection accuracy for small objects (such as augmentation dots and accidentals) while maintaining real-time performance.

Implementation Details: The YOLOv12 model is trained for 100 epochs with SGD optimizer. Input resolution is set to 640. The YOLO model is trained to minimize a composite loss function including bounding box regression and classification error. We utilize Mosaic data augmentation to enhance the model's robustness against scale variations and partial occlusions.

Results: The YOLOv12 model achieves high mAP scores on common symbols (Noteheads, Clefs), ensuring a solid foundation for subsequent assembly stages.

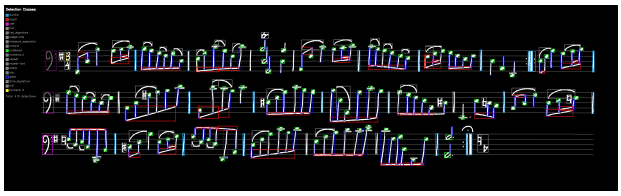

Figure 4: YOLOv12 detection results with bounding boxes around identified musical symbols.

2.4 Relation Prediction (MLP Linker)

After detecting discrete symbols, the system must reconstruct their relationships (e.g., which notehead is attached to which stem). We model this as a link prediction problem on a fully connected graph where each detected symbol is a node.

Feature Engineering: For a pair of symbols (u, v), the model extracts a comprehensive feature vector combining geometric and semantic information:

- **Geometric Features**: Relative distance ($d_x = x_v - x_u$, $d_y = y_v - y_u$), normalized bounding box coordinates, and overlap ratios including IoU (Intersection over Union). These features capture the spatial relationships critical for determining valid connections.
- **Semantic Features**: Learned embedding vectors $\mathbf{e}_u, \mathbf{e}_v \in \mathbb{R}^{32}$ corresponding to the class IDs of symbols u and v. The embeddings are concatenated as $[\mathbf{e}_u; \mathbf{e}_v]$ to form a 64-dimensional semantic representation, allowing the model to learn class-specific connection patterns (e.g., noteheads typically connect to stems, but not to clefs).

The MLP outputs a probability score $P(u \to v) \in [0, 1]$ indicating the likelihood of a directed edge existing between them. This allows us to construct a **Notation Graph** where nodes are primitives and edges represent semantic connections.

Implementation Details: The MLP consists of 3 hidden layers with ReLU activation. It is trained to predict adjacency matrices from symbol pairs.

Evaluation Metrics: We adopt the **Match+AUC** metric proposed by Yang et al. [4] to evaluate the MLP Linker's performance. This metric combines two complementary aspects:

- **Match Score**: Measures the precision of predicted edges against ground truth notation graphs. For each predicted edge (u, v), we verify if it exists in the ground truth graph, computing the ratio of correct predictions.
- **AUC (Area Under Curve)**: Evaluates the ranking quality of edge probabilities. A higher AUC indicates better discrimination between true and false edges across different threshold values.

Results: The MLP Linker achieves a **Match+AUC score of 0.86** on the test set, demonstrating that our model effectively learns the geometric and semantic patterns of symbol relationships. It successfully filters out valid connections from the quadratic number of possible pairs. The conceptual "overfitting" ensures that for the limited vocabulary of a single page, the connectivity is robust.

2.5 Semantic Assembly

This module transforms the Notation Graph into a hierarchical **Score Tree** for MusicXML generation. The assembly pipeline consists of six sequential phases.

2.5.1 System & Staff Grouping

We first partition staff lines into *Systems* (musical rows) using a priority-based strategy: symbols like `brace` or `measure_separator` group vertically overlapping staves ($Overlap_Y > 0.5$). Within each system, staves are sorted top-to-bottom and assigned sequential Part IDs.

2.5.2 Measure Slicing

To segment time horizontally, we detect barlines using geometric validation (aspect ratio < 0.3, minimum height ≥ 20px). Since detection may produce slightly misaligned barlines at the same temporal position, we merge those within threshold $\max(\frac{w_{notehead}}{2}, 15)$ pixels by averaging their X-coordinates. The resulting *Global Barlines* define measure boundaries via bucket sort.

2.5.3 Symbol Assembly

Musical notes are constructed by linking primitives. For filled noteheads without overlapping stems, we create *virtual stems*: we search upward/downward for beams or flags, extending the virtual stem to reach them if found, or creating a default-length stem ($3.5 \times line\_spacing$) otherwise. Stem direction is determined by the notehead's position relative to the middle staff line. Each stem connects to all nearby beams/flags within $stem.width$ distance. Dotted notes are detected by searching rightward for `duration-dot` symbols, multiplying the duration by 1.5.

2.5.4 Attribute Recognition

Musical attributes (clef, key signature, time signature) exhibit **state persistence**: they remain active until explicitly changed. For each measure, we search for attribute symbols; if absent, we inherit from the previous measure. Key signatures are identified by clustering `sharp`/`flat` symbols with spacing $< 1.5 \times note\_width$.

2.5.5 Tuplet Detection

We employ a cascaded rule set with decreasing constraint strength. **Rule 1** searches for `tuple_bracket` symbols covering exactly 3 notes. **Rule 2** identifies beams linking $3k$ stems with a `numeral_3` nearby, excluding finger

numbers (vertically aligned above a single note). **Rule 3** uses Euclidean distance to find the 3 closest notes to isolated `numeral_3` symbols. **Rule 4** applies the triplet modification: $duration_{xml} = base\_duration \times \frac{2}{3}$. **Rule 5** serves as a fallback sanity check: if the measure duration exceeds the time signature, we search for groups of 3 identical-duration notes that would fix the overflow.

2.5.6 Pitch Determination

Pitch is calculated from the notehead's vertical position. We find the closest staff line, compute the step distance using half-spacing $h = median(\Delta y_{lines})/2$, and apply the formula:

$$steps = i_{closest} \times 2 + \frac{y_{note} - y_{closest}}{h}$$

This value is mapped to a diatonic pitch using clef-specific references (Treble: top line = F5; Bass: A3; Alto: G4). Accidentals are then applied to adjust the base pitch.

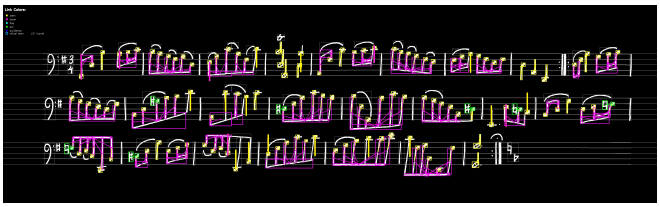

Figure 5: Semantic assembly result showing reconstructed relationships between musical symbols.

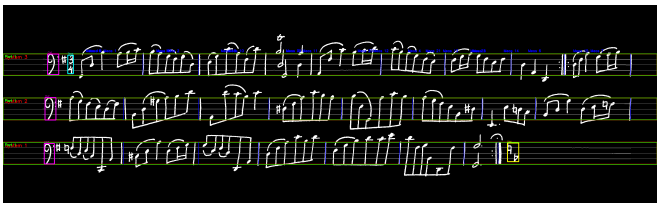

Figure 6: System structure visualization showing grouped staff systems and measure boundaries.

2.6 MusicXML Generation

The final Assembler successfully generates MusicXML files that can be opened in MuseScore and other music notation software. The system correctly reconstructs polyphonic voices and complex tuplets using the defined rules. Figure 1 shows the complete data flow from input image to final MusicXML output.

3 Conclusion and Future Work

Our system demonstrates the powerful potential of combining computer vision with symbolic logic. By treating sheet music as a graph structure, we effectively captured the complex two-dimensional dependencies between musical symbols, successfully achieving the digital reconstruction of handwritten music.

An improvement direction can be following areas:

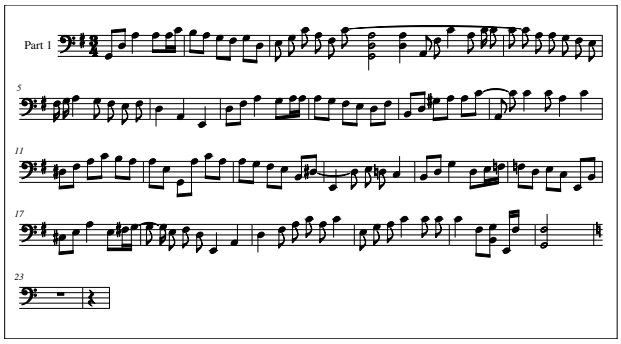

Figure 7: Final MusicXML output rendered as musical notation, demonstrating successful end-to-end reconstruction.

- **improve linker model**: The current linker model shows a high benchmark performance, but less useful in data due to illogical mismatches. And current rule-based reasoning is not enough to handle complex cases. We can try to use small models to help some subprocesses of rule-based reasoning.
- **More Symbol Types**: So far we only consider the essential symbols, we will try to include more symbols to improve the performance.
- **porbabilitistic and interactive processing**: Here we use yolo to detect the symbols with a threshold, but we can implement an interface that allows users to modify the recognition results with a probability output like Audervis.

References

[1] María Alfaro-Contreras, José M. Iñesta, and Jorge Calvo-Zaragoza. Optical music recognition for homophonic scores with neural networks and synthetic music generation. *International Journal of Multimedia Information Retrieval*, 12(1):12, May 2023.

[2] Jorge Calvo-Zaragoza, Jan Hajič Jr., and Alexander Pacha. Understanding optical music recognition. *ACM Computing Surveys*, 53(4):1–35, July 2020.

[3] Jiří Mayer, Milan Straka, Jan Hajič, and Pavel Pecina. *Practical End-to-End Optical Music Recognition for Pianoform Music*, page 55–73. Springer Nature Switzerland, 2024.

[4] Guang Yang, Muru Zhang, Lin Qiu, Yanming Wan, and Noah A. Smith. Toward a more complete omr solution, 2024.